

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Artificial Intelligence

COURSE CODE: CSA17

COURSE OUTCOME COVERED

CO4: *Implement machine learning techniques including decision tree algorithms, reinforcement learning, and build models for classification and decision-making. (BL3)*

Assessment Tool Weightage: 40%

QUESTIONS: 2

Duration : 45 Minutes
Total Marks:20

Annexure A

INTEGRATED EXPERIMENTS

Code of Conduct:

I K.R.Jeevan Balaji(192471032) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:

INTEGRATED EXPERIMENTS

Experiment 1: Decision Tree Classification

Objective: Implement and evaluate a Decision Tree classifier on a given dataset (e.g., Iris / Play-Tennis) using Python / any ML library.

- (a) Load the dataset, pre-process it (handle missing values, encoding), and split into training and testing sets. Display the first 5 rows and data types.
- (b) Build a Decision Tree model using Gini Index / Information Gain as the splitting criterion. Print the tree structure and identify the root node with justification
- (c) Evaluate the model using accuracy score, confusion matrix, and classification report. Comment on the performance and list at least two misclassified instances.

Experiment 2: Reinforcement Learning – Q-Learning

Objective: Implement Q-Learning for a simple Grid World (4×4) environment and observe the agent's learned policy.

- (a) Define the environment: states, actions (Up/Down/Left/Right), reward structure (+10 Goal, -1 each step, -5 obstacle). Initialize the Q-table with zeros and state the hyperparameters (α , γ , ϵ).
- (b) Run the Q-Learning algorithm for at least 100 episodes. Record the Q-values at Episodes 1, 50, and 100 for two representative states. Show how the Q-table evolves.
- (c) Extract and display the final learned policy (optimal action at each state). Plot the cumulative reward per episode and justify the convergence behaviour.

ANSWERS

1)

Experiment 1: Decision Tree Classification

Objective

To implement and evaluate a Decision Tree classifier on the Iris dataset using Python and Machine Learning libraries.

(a) Load, Pre-process and Split the Dataset

```
import pandas as pd
from sklearn.datasets import load_iris
from sklearn.model_selection import train_test_split

# Load Iris dataset
iris = load_iris()

# Create DataFrame
df = pd.DataFrame(iris.data, columns=iris.feature_names)
df['target'] = iris.target

# Display first 5 rows
print("First 5 rows:")
print(df.head())

# Display data types
print("\nData Types:")
print(df.dtypes)

# Check missing values
print("\nMissing Values:")
print(df.isnull().sum())

# Separate features and target
X = df.drop('target', axis=1)
y = df['target']

# Split into training and testing data
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

print("\nTraining samples:", X_train.shape[0])
print("Testing samples:", X_test.shape[0])
```

Output

The first five rows will contain:

Sepal Length	Sepal Width	Petal Length	Petal Width	Target
5.1	3.5	1.4	0.2	0
4.9	3.0	1.4	0.2	0
4.7	3.2	1.3	0.2	0
4.6	3.1	1.5	0.2	0
5.0	3.6	1.4	0.2	0

There are no missing values, so no missing-value treatment is required. The Iris dataset contains numerical features, so encoding is also not required.

(b) Build the Decision Tree

Here we use Gini Index as the splitting criterion.

```
from sklearn.tree import DecisionTreeClassifier
from sklearn.tree import export_text, plot_tree
import matplotlib.pyplot as plt

# Create Decision Tree classifier using Gini Index
model = DecisionTreeClassifier(
    criterion='gini',
    random_state=42
)

# Train the model
model.fit(X_train, y_train)

# Print tree structure
tree_rules = export_text(
    model,
    feature_names=list(X.columns)
)

print("Decision Tree Structure:")
print(tree_rules)

# Print root node information
root_feature_index = model.tree_.feature[0]
root_threshold = model.tree_.threshold[0]

print("\nRoot Node Feature:", X.columns[root_feature_index])
print("Root Node Threshold:", root_threshold)

# Visualize the tree
plt.figure(figsize=(15, 8))
plot_tree(
    model,
    feature_names=X.columns,
    class_names=iris.target_names,
    filled=True
)
plt.show()
```

Root Node Identification

For the Iris dataset, the Decision Tree generally selects:

Petal Length (cm)

as the root node when using this train/test split and `random_state=42`.

The root node is selected because it provides the largest reduction in impurity, i.e., the best Gini-based split among the available features.

For example, the tree may begin with a rule similar to:

```
petal length (cm) <= 2.45
```

This separates Iris-setosa very effectively from the other two classes.

Justification

The Gini Index measures the impurity of a node:

$$\text{Gini} = 1 - \sum p_i^2$$

A lower Gini value indicates a purer node. The Decision Tree evaluates possible features and thresholds and chooses the split that gives the greatest reduction in impurity. Therefore, the feature selected at the root is the most effective feature for separating the classes.

(c) Model Evaluation

```
from sklearn.metrics import (
    accuracy_score,
    confusion_matrix,
    classification_report
)

# Predict test data
y_pred = model.predict(X_test)

# Accuracy
accuracy = accuracy_score(y_test, y_pred)

print("Accuracy:", accuracy)

# Confusion Matrix
cm = confusion_matrix(y_test, y_pred)

print("\nConfusion Matrix:")
print(cm)

# Classification Report
print("\nClassification Report:")
print(classification_report(
    y_test,
    y_pred,
    target_names=iris.target_names
))
```

Typical Output

You should obtain an accuracy close to:

Accuracy: 0.9333

or another high value depending on the tree and split configuration.

A typical confusion matrix may look like:

```
[[10  0  0]
 [ 0  9  1]
 [ 0  1  9]]
```

This means:

- 10 Setosa samples were correctly classified.
- 9 Versicolor samples were correctly classified.
- 9 Virginica samples were correctly classified.
- One Versicolor was classified as Virginica.
- One Virginica was classified as Versicolor.

Classification Report

The classification report provides:

- Precision – how many predicted samples belong to the correct class.
- Recall – how many actual samples were correctly identified.
- F1-score – harmonic mean of precision and recall.
- Support – number of samples belonging to each class.

Finding Misclassified Instances

Use the following code to display the actual misclassified samples:

```
# Find misclassified instances
misclassified = X_test[y_test != y_pred].copy()

misclassified['Actual'] = y_test[y_test != y_pred]
misclassified['Predicted'] = y_pred[y_test != y_pred]

# Convert class numbers to class names
misclassified['Actual'] = misclassified['Actual'].map(
    dict(enumerate(iris.target_names))
)

misclassified['Predicted'] = misclassified['Predicted'].map(
    dict(enumerate(iris.target_names))
)

print("\nMisclassified Instances:")
print(misclassified)
```

Performance Comment

The Decision Tree performs well on the Iris dataset because the three flower species have distinctive measurements, particularly petal length and petal width. Setosa is usually classified almost perfectly, while a small number of Versicolor and Virginica samples may be confused because their measurements overlap.

Thus, the model achieves high accuracy with relatively few misclassifications, indicating that Decision Tree classification is effective for this dataset.

Result

The Decision Tree classifier was successfully implemented using the Iris dataset. The data was pre-processed and divided into training and testing sets. A Decision Tree using the Gini Index was trained, its structure and root node were identified, and its performance was evaluated using accuracy, confusion matrix, classification report, and misclassified instances.

2)

Experiment 2: Reinforcement Learning – Q-Learning

Objective

To implement the Q-Learning algorithm for a simple 4×4 Grid World environment and observe the agent's learned optimal policy.

(a) Define the Environment

We consider a 4×4 Grid World with 16 states.

```
+-----+-----+-----+-----+
| S0 | S1 | S2 | S3 |
+-----+-----+-----+-----+
| S4 | S5 | S6 | S7 |
+-----+-----+-----+-----+
| S8 | S9 | S10| S11|
+-----+-----+-----+-----+
| S12| S13| S14| G  |
+-----+-----+-----+-----+
```

- Start state: S0
- Goal state: S15
- Actions: Up, Down, Left, Right
- Goal reward: +10
- Normal step: -1
- Obstacle: -5
- Q-table: Initially filled with zeros.

Hyperparameters

Parameter	Value	Meaning
α	0.1	Learning rate
γ	0.9	Discount factor
ϵ	0.1	Exploration rate
Episodes	100	Number of training episodes

The Q-learning update equation is:

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$

Python Program

```
import numpy as np
import random
import matplotlib.pyplot as plt

# Grid size
ROWS = 4
COLS = 4
STATES = ROWS * COLS

# Actions
ACTIONS = ['Up', 'Down', 'Left', 'Right']
NUM_ACTIONS = len(ACTIONS)

# Hyperparameters
alpha = 0.1 # Learning rate
gamma = 0.9 # Discount factor
```

```

epsilon = 0.1      # Exploration rate
episodes = 100

# Goal and obstacle
goal = 15
obstacles = {5, 10}

# Initialize Q-table
Q = np.zeros((STATES, NUM_ACTIONS))

# Starting state
start_state = 0

# Function to convert state to row and column
def get_position(state):
    return state // COLS, state % COLS

# Function to get next state and reward
def step(state, action):
    row, col = get_position(state)

    # Move according to action
    if action == 0:      # Up
        new_row = max(0, row - 1)
        new_col = col

    elif action == 1:    # Down
        new_row = min(ROWS - 1, row + 1)
        new_col = col

    elif action == 2:    # Left
        new_row = row
        new_col = max(0, col - 1)

    else:                # Right
        new_row = row
        new_col = min(COLS - 1, col + 1)

    next_state = new_row * COLS + new_col

    # Reward structure
    if next_state == goal:
        reward = 10
        done = True

    elif next_state in obstacles:
        reward = -5
        done = False

    else:
        reward = -1
        done = False

    return next_state, reward, done

```

(b) Run Q-Learning for 100 Episodes

The agent chooses actions using the ϵ -greedy strategy.

- With probability $\epsilon \rightarrow$ choose a random action.
- Otherwise $\rightarrow$ choose the action having the highest Q-value.

```
# Store cumulative rewards
episode_rewards = []

# Store Q-values at specific episodes
q_history = {}

for episode in range(1, episodes + 1):

    state = start_state
    total_reward = 0

    for step_count in range(100):

        # Epsilon-greedy action selection
        if random.random() < epsilon:
            action = random.randint(0, NUM_ACTIONS - 1)
        else:
            action = np.argmax(Q[state])

        # Take action
        next_state, reward, done = step(state, action)

        # Q-learning update
        Q[state, action] = Q[state, action] + alpha * (
            reward +
            gamma * np.max(Q[next_state]) -
            Q[state, action]
        )

        total_reward += reward

        state = next_state

        if done:
            break

    episode_rewards.append(total_reward)

# Record Q-values at episodes 1, 50 and 100
if episode in [1, 50, 100]:
    q_history[episode] = Q.copy()

# Display Q-values for two representative states
for episode in [1, 50, 100]:

    print("\nEpisode", episode)

    print("State S0:", q_history[episode][0])
    print("State S6:", q_history[episode][6])
```

Observing Q-table Evolution

The Q-values initially contain zeros:

Episode 1

```
S0: [0. 0. 0. 0.]
S6: [0. 0. 0. 0.]
```

As the agent explores the environment, the Q-values are updated.

A typical progression is:

	S0 Q-values	S6 Q-values
Episode 1	[small values]	[small values]
Episode 50	[learned values]	[learned values]
Episode 100	[stable values]	[stable values]

The exact numerical values can vary because Q-learning uses random exploration.

Display Complete Q-table

```
print("\nFinal Q-Table:")
print(Q)
```

The final Q-table contains four Q-values for every state:

State	Up	Down	Left	Right
S0	...	...	...	...
S1	...	...	...	...
S2	...	...	...	...
...				
S15	...	...	...	...

(c) Extract the Final Learned Policy

The learned policy is obtained by selecting the action having the maximum Q-value for every state.

```
# Extract final policy
policy = np.argmax(Q, axis=1)

print("\nFinal Learned Policy:")

for state in range(STATES):

    if state == goal:
        print(f"S{state}: GOAL")

    elif state in obstacles:
        print(f"S{state}: OBSTACLE")

    else:
        print(f"S{state}: {ACTIONS[policy[state]]}")
```

Display Policy as a Grid

```
print("\nPolicy Grid:")

symbols = {
    'Up': '↑',
    'Down': '↓',
    'Left': '←',
    'Right': '→'
}

for row in range(ROWS):
```

```

for col in range(COLS):

    state = row * COLS + col

    if state == goal:
        symbol = 'G'

    elif state in obstacles:
        symbol = 'X'

    else:
        action = ACTIONS[policy[state]]
        symbol = symbols[action]

    print(f"{symbol:^5}", end='')

print()

```

A typical learned policy will look approximately like:

```

+-----+-----+-----+-----+
|  →  |  →  |  →  |  ↓  |
+-----+-----+-----+-----+
|  ↓  |  X  |  →  |  ↓  |
+-----+-----+-----+-----+
|  →  |  →  |  X  |  ↓  |
+-----+-----+-----+-----+
|  →  |  →  |  →  |  G  |
+-----+-----+-----+-----+

```

The exact arrows may differ depending on the random training process.

Plot Cumulative Reward

```

plt.figure(figsize=(10, 5))

plt.plot(episode_rewards)

plt.xlabel("Episode")
plt.ylabel("Cumulative Reward")
plt.title("Cumulative Reward per Episode")
plt.grid(True)

plt.show()

```

Convergence Behaviour

Initially, the agent has no knowledge about the environment because all Q-values are zero. Therefore, it explores different actions and receives several negative rewards.

As training progresses:

1. The agent discovers the goal state.
2. Positive reward information propagates backward through the Q-table.
3. Actions leading toward the goal receive higher Q-values.
4. The agent gradually avoids unnecessary movements and obstacles.
5. The cumulative reward generally improves and becomes more stable.

Therefore, the Q-learning algorithm converges toward an optimal or near-optimal policy after sufficient episodes.

Result

The Q-Learning algorithm was successfully implemented for a 4×4 Grid World. The Q-table was initialized with zeros and updated for 100 episodes using the Q-learning update rule. The Q-values at different stages demonstrated the learning process. Finally, the learned policy was extracted from the Q-table, and the cumulative reward plot demonstrated the convergence of the agent toward a better path to the goal.

RUBRICS FOR CALCULATION:

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Concepts	25	Demonstrates thorough understanding and effectively integrates multiple concepts/technologies	Good understanding with proper integration of most concepts	Basic understanding; limited or partial integration	Poor understanding; unable to integrate concepts
Experimental Design	30	Well-planned, systematic implementation with optimal solution	Correct implementation with minor issues	Basic implementation with errors or inefficiencies	Incorrect or incomplete implementation

Use of Tools	20	Effective and advanced use of tools/technologies with proper configuration	Appropriate use of tools with correct execution	Limited or inconsistent use of tools	Incorrect or minimal use of tools
Analysis of Results	15	Insightful analysis with accurate interpretation and validation of results	Good analysis with reasonable interpretation	Basic analysis with limited insights	Poor or incorrect analysis of results
Documentation	10	Clear, well-structured documentation with diagrams, code, and results	Organized documentation with necessary details	Basic documentation with missing details	Poor or incomplete documentation